# Data Structures & Algorithms — Python Handbook

A structured reference for study and revision. Export this document to PDF for offline use.

## Table of Contents


1. Arrays
2. Strings
3. Linked List
4. Stack
5. Queue
6. Recursion
7. Sorting Algorithms
8. Searching Algorithms
9. Trees
10. Graphs
11. Dynamic Programming
12. Miscellaneous Important Problems


## 1. Arrays

### 1.1 Maximum Subarray Sum (Kadane's Algorithm)

| | |
|---|---|
| **Concept** | Array, Dynamic Programming (Kadane's Algorithm) |
| **Problem** | Given an array of integers, find the contiguous subarray (at least one element) with the largest sum and return that sum. |

#### Approach / Logic

1. If the array is empty, return 0.

2. Keep two variables: `max_sum` (best sum so far) and `current_sum` (sum of current subarray ending at this element).
3. For each element: either start a new subarray at this element, or extend the previous subarray: `current_sum = max(num, current_sum + num)` .
4. Update `max_sum` with the maximum of `max_sum` and `current_sum` .
5. Return `max_sum` .

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```python
def max_subarray(nums):
    if not nums:
        return 0
    max_sum = nums[0]
    current_sum = nums[0]
    for num in nums[1:]:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)
    return max_sum
```

### Example

- Input: `[-2, 1, -3, 4, -1, 2, 1, -5, 4]`
- Output: `6` (subarray `[4, -1, 2, 1]` )

**Takeaways:** One pass; no extra array. Classic DP pattern: “best ending here” then “best so far”.

## 1.2 Rotate Array to the Right

| | |
|---|---|
| **Concept** | Array, In-place modification |
| **Problem** | Given a list of integers and non-negative `k` , rotate the list to the right by `k` steps in-place. |

### Approach / Logic

1. Reduce `k` with `k = k % len(nums)` to handle `k` larger than length.
2. Right rotation by `k` means the last `k` elements move to the front: `nums[:] = nums[-k:] + nums[:-k]` .

3. Use slice assignment so the original list is modified in-place.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(n) for slice copy |

```
def rotate(nums, k):
    if not nums:
        return
    k = k % len(nums)
    nums[:] = nums[-k:] + nums[:-k]
```

### Example

- Input: `nums = [1, 2, 3, 4, 5, 6, 7]`, `k = 3`
- Output (in-place): `[5, 6, 7, 1, 2, 3, 4]`

**Takeaways:** Modulo handles `k > n`. For true O(1) space, use three reversals (reverse whole, reverse first k, reverse rest).

## 1.3 Best Time to Buy and Sell Stock (One Transaction)

| | |
|---|---|
| **Concept** | Array, Greedy |
| **Problem** | Given daily stock prices, find the maximum profit from one buy and one sell (buy before sell). |

### Approach / Logic

1. Track the minimum price seen so far and the maximum profit so far.
2. For each day: update min price, then profit = price - min_price; update max profit.
3. Return max profit.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```
def max_profit(prices):
    if not prices:
        return 0
    min_price = prices[0]
```

```
    max_profit = 0
    for price in prices:
        min_price = min(min_price, price)
        profit = price - min_price
        max_profit = max(max_profit, profit)
    return max_profit
```

### Example

- Input: `[7, 1, 5, 3, 6, 4]`
- Output: `5` (buy at 1, sell at 6)

**Takeaways:** One pass; always consider “best so far” and “minimum so far”.

## 1.4 Remove Duplicates from Sorted Array (In-Place)

| | |
|---|---|
| **Concept** | Array, Two pointers |
| **Problem** | Given a sorted list, move all unique elements to the front in-place and return the new length. |

### Approach / Logic

1. Use index `i` for the position of the last unique element (start at 0).
2. Iterate with `j` from 1 to end. When `nums[j] != nums[i]`, increment `i` and set `nums[i] = nums[j]`.
3. Return `i + 1` as the new length.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```
def remove_duplicates(nums):
    if not nums:
        return 0
    i = 0
    for j in range(1, len(nums)):
        if nums[j] != nums[i]:
            i += 1
            nums[i] = nums[j]
    return i + 1
```

### Example

- Input: `[0, 0, 1, 1, 1, 2, 2, 3, 3, 4]`
- Output: `5` ; first 5 elements become `[0, 1, 2, 3, 4, ...]`

**Takeaways:** Two-pointer in-place update; works only when input is sorted.

## 1.5 Find Maximum and Minimum in Array

| | |
|---|---|
| **Concept** | Array, Linear scan |
| **Problem** | Find the maximum and minimum values in an array in one pass. |

### Approach / Logic

1. Track indices (or values) of current min and max; initialize with index 0.
2. From index 1 to end, update min_index if element is smaller, max_index if larger.
3. Return `(max_value, min_value)` .

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```python
def find_max_min(arr):
    if not arr:
        return None, None
    min_idx = 0
    max_idx = 0
    for i in range(1, len(arr)):
        if arr[i] < arr[min_idx]:
            min_idx = i
        if arr[i] > arr[max_idx]:
            max_idx = i
    return arr[max_idx], arr[min_idx]
```

### Example

- Input: `[5, 3, 8, 1, 6, 9]`
- Output: `(9, 1)`

**Takeaways:** Single pass; 2 comparisons per element.

### 1.6 Remove All Occurrences of Value (In-Place)

| | |
|---|---|
| **Concept** | Array, In-place, Index control |
| **Problem** | Remove all occurrences of `val` from the list in-place and return the new length. |

#### Approach / Logic

1. Use a `while` loop with index `i` so we control when to advance (don't advance when we pop).
2. If `nums[i] == val`, pop at `i` (list shrinks; next element moves to `i`).
3. Otherwise, increment `i`.
4. Return `len(nums)`.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```
def remove_element(nums, val):
    i = 0
    while i < len(nums):
        if nums[i] == val:
            nums.pop(i)
        else:
            i += 1
    return len(nums)
```

**Takeaways:** Avoid iterating with `for` + `pop`; use `while` and only increment when not removing.

## 2. Strings

### 2.1 Find Longest String in List

| | |
|---|---|
| **Concept** | String, List, Linear scan |
| **Problem** | Given a list of strings, return the string with the maximum length. |

#### Approach / Logic

1. Initialize `longest` as empty string (or first element).

2. For each string, if its length is greater than `longest` 's length, update `longest` .
3. Return `longest` .

| Time Complexity | Space Complexity |
|---|---|
| O(n * m) | O(1) |

*n = number of strings, m = max string length*

```
def find_longest_string(string_list):
    longest = ""
    for s in string_list:
        if len(s) > len(longest):
            longest = s
    return longest
```

### Example

- Input: `['apple', 'banana', 'kiwi', 'pear']`
- Output: `'banana'`

**Takeaways:** Single pass; no extra data structure.

## 2.2 Reverse String Using Stack

| | |
|---|---|
| **Concept** | String, Stack (LIFO) |
| **Problem** | Reverse a string using a stack. |

### Approach / Logic

1. Push each character onto the stack.
2. Pop characters one by one and append to result string (LIFO gives reverse order).
3. Return the result.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(n) |

```
def reverse_string(s):
    stack = []
```

```
    for char in s:
        stack.append(char)
    result = ""
    while stack:
        result += stack.pop()
    return result
```

### Example

- Input: `'hello'`
- Output: `'olleh'`

**Takeaways:** Stack naturally reverses order; same idea applies to reversing linked lists with recursion.

## 2.3 Balanced Parentheses

| | |
|---|---|
| **Concept** | String, Stack |
| **Problem** | Check if a string of parentheses is balanced (every `(` has a matching `)`). |

### Approach / Logic

1. For each character: if `(`, push; if `)`, pop (if stack empty, return False).
2. At the end, return True only if the stack is empty.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(n) |

```
def is_balanced_parentheses(s):
    stack = []
    for char in s:
        if char == '(':
            stack.append(char)
        elif char == ')':
            if not stack:
                return False
            stack.pop()
    return len(stack) == 0
```

### Example

- `'((()))'` → True; `'(()'` → False; `'())'` → False

**Takeaways:** Stack matches last opened with current closing; extends to multiple bracket types with a mapping.

### 2.4 First Non-Repeating Character

| | |
|---|---|
| **Concept** | String, Hash table (counts) |
| **Problem** | Find the first character in a string that appears exactly once; return None if none. |

#### Approach / Logic

1. One pass: count each character in a dictionary.
2. Second pass (in original order): return the first character with count 1. (In Python 3.7+, dict preserves insertion order, so one pass over the string when building counts is enough; then iterate dict or string to find first with count 1.)

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) if alphabet fixed |

```python
def first_non_repeating_char(s):
    counts = {}
    for char in s:
        counts[char] = counts.get(char, 0) + 1
    for char in s:
        if counts[char] == 1:
            return char
    return None
```

#### Example

- `'leetcode'` → `'l'`; `'hello'` → `'h'`; `'aabbcc'` → `None`

**Takeaways:** Order matters; iterate the string (not just the dict) to get “first”.

## 3. Linked List

### 3.1 Singly Linked List — Basic Implementation

| | |
|---|---|
| **Concept** | Singly Linked List, Nodes with `value` and `next` |
| **Problem** | Implement a singly linked list with append, pop, prepend, pop_first, get, set_value, insert, remove, and reverse. |

### Approach / Logic

- **Node:** `value` , `next` .
- **Append:** Traverse to tail, set `tail.next = new_node` , update tail.
- **Pop:** Traverse to second-last, set its `next = None` , update tail.
- **Prepend:** `new_node.next = head` , update head.
- **Pop first:** `head = head.next` , detach old head.
- **Get(index):** Traverse from head `index` steps.
- **Insert:** Get node at index-1, link new node between it and its next.
- **Remove:** Get node at index-1, bypass the node at index.
- **Reverse:** Iterate with `before` , `temp` , `after` ; reverse each link; finally swap head and tail.

| Time Complexity | Space Complexity |
|---|---|
| Access by index O(n), Append O(1) with tail | O(n) for list |

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class LinkedList:
    def __init__(self, value):
        new_node = Node(value)
        self.head = new_node
        self.tail = new_node
        self.length = 1

    def append(self, value):
        new_node = Node(value)
        if self.length == 0:
            self.head = self.tail = new_node
        else:
            self.tail.next = new_node
            self.tail = new_node
        self.length += 1

    def pop(self):
```

```
        if self.length == 0:
            return None
        temp = self.head
        prev = self.head
        while temp.next:
            prev, temp = temp, temp.next
        self.tail = prev
        self.tail.next = None
        self.length -= 1
        if self.length == 0:
            self.head = self.tail = None
        return temp.value

    def prepend(self, value):
        new_node = Node(value)
        if self.length == 0:
            self.head = self.tail = new_node
        else:
            new_node.next = self.head
            self.head = new_node
        self.length += 1
        return True

    def pop_first(self):
        if self.length == 0:
            return None
        temp = self.head
        self.head = self.head.next
        temp.next = None
        self.length -= 1
        if self.length == 0:
            self.tail = None
        return temp.value

    def get(self, index):
        if index < 0 or index >= self.length:
            return None
        temp = self.head
        for _ in range(index):
            temp = temp.next
        return temp

    def set_value(self, index, value):
        node = self.get(index)
        if node:
            node.value = value
            return True
        return False
```

```
    def insert(self, index, value):
        if index < 0 or index > self.length:
            return False
        if index == 0:
            return self.prepend(value)
        if index == self.length:
            self.append(value)
            return True
        new_node = Node(value)
        prev = self.get(index - 1)
        new_node.next = prev.next
        prev.next = new_node
        self.length += 1
        return True

    def remove(self, index):
        if index < 0 or index >= self.length:
            return None
        if index == 0:
            return self.pop_first()
        if index == self.length - 1:
            return self.pop()
        prev = self.get(index - 1)
        temp = prev.next
        prev.next = temp.next
        temp.next = None
        self.length -= 1
        return temp.value

    def reverse(self):
        temp = self.head
        self.head = self.tail
        self.tail = temp
        before = None
        for _ in range(self.length):
            after = temp.next
            temp.next = before
            before = temp
            temp = after
```

**Takeaways:** Maintain `head`, `tail`, and `length`; reverse in one pass by reversing each link.

## 3.2 Find Kth Node From End (Two Pointers)

| | |
|---|---|
| **Concept** | Linked List, Two pointers |

| | |
|---|---|
| **Problem** | Return the k-th node from the end without using the list length (1 = last node). |

### Approach / Logic

1. Move `fast` forward by `k` steps; if `fast` becomes None before that, return None.
2. Move `slow` and `fast` together until `fast` reaches the last node.
3. `slow` is then the k-th node from the end.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```
def find_kth_from_end(linked_list, k):
    slow = fast = linked_list.head
    for _ in range(k):
        if fast is None:
            return None
        fast = fast.next
    while fast:
        slow = slow.next
        fast = fast.next
    return slow
```

### Example

- List: `1 -> 2 -> 3 -> 4 -> 5` , `k = 2` → node with value `4`

**Takeaways:** “Runner” technique: k-step lead gives k-th from end when fast hits the end.

## 3.3 Remove Duplicates from Linked List (Hash Set)

| | |
|---|---|
| **Concept** | Linked List, Hash set |
| **Problem** | Remove all nodes with duplicate values; keep one occurrence of each value. |

### Approach / Logic

1. Use a set of seen values; maintain `prev` and `current` .
2. If `current.value` is in set, skip node: `prev.next = current.next` , decrement length.
3. Otherwise add value to set, set `prev = current` .

4. Advance `current`.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(n) |

```
def remove_duplicates(self):
    if not self.head:
        return
    seen = set()
    prev = None
    current = self.head
    while current:
        if current.value in seen:
            prev.next = current.next
            self.length -= 1
        else:
            seen.add(current.value)
            prev = current
        current = current.next
```

**Takeaways:** Hash set for "seen"; prev pointer to unlink duplicates.

## 3.4 Reverse Linked List Between Indices

| | |
|---|---|
| **Concept** | Linked List, Dummy node, In-place reversal |
| **Problem** | Reverse the sublist from index `start` to index `end` (inclusive). |

### Approach / Logic

1. Use a dummy node before head to handle `start == 0`.
2. Move `prev` to the node just before the sublist (after `start` steps).
3. Reverse the sublist by repeatedly moving the next node to the front of the sublist (end - start times).
4. Update head from dummy if needed.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```
def reverse_between(self, start_index, end_index):
    if self.length <= 1:
        return
    dummy = Node(0)
    dummy.next = self.head
    prev = dummy
    for _ in range(start_index):
        prev = prev.next
    current = prev.next
    for _ in range(end_index - start_index):
        node_to_move = current.next
        current.next = node_to_move.next
        node_to_move.next = prev.next
        prev.next = node_to_move
    self.head = dummy.next
```

**Takeaways:** Dummy node simplifies edge cases; “move next to front” gives in-place reversal of a segment.

## 3.5 Partition List Around Value

| | |
|---|---|
| **Concept** | Linked List, Two dummy nodes |
| **Problem** | Partition the list so all nodes with value < x come before nodes with value ≥ x; preserve relative order within each part. |

### Approach / Logic

1. Create two dummy heads: one for “less than x”, one for “≥ x”.
2. Traverse the list: append each node to the appropriate list.
3. Connect the “less” list to the “greater or equal” list; set next of last node of second list to None.
4. Update head to dummy_less.next.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```
def partition_list(self, x):
    if not self.head:
        return
```

```
        dummy_less = Node(0)
        dummy_ge = Node(0)
        prev_less, prev_ge = dummy_less, dummy_ge
        current = self.head
        while current:
            if current.value < x:
                prev_less.next = current
                prev_less = current
            else:
                prev_ge.next = current
                prev_ge = current
            current = current.next
        prev_less.next = dummy_ge.next
        prev_ge.next = None
        self.head = dummy_less.next
```

**Takeaways:** Two lists then concatenate; no extra node allocation for values.

## 3.6 Palindrome Linked List (Singly)

| | |
|---|---|
| **Concept** | Linked List, Slow-fast pointers, Reversal |
| **Problem** | Determine if a singly linked list is a palindrome. |

### Approach / Logic

1. Find middle with slow/fast pointers (slow moves 1, fast moves 2).
2. Reverse the second half starting from slow.
3. Compare first half (from head) with reversed second half (from prev).
4. Return True if all match.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```
def is_palindrome(self):
    if not self.head or not self.head.next:
        return True
    slow = fast = self.head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    prev = None
```

```
        current = slow
        while current:
            nxt = current.next
            current.next = prev
            prev = current
            current = nxt
        first, second = self.head, prev
        while second:
            if first.value != second.value:
                return False
            first, second = first.next, second.next
        return True
```

**Takeaways:** Find middle, reverse second half, compare; O(1) space if we don't count recursion.

## 3.7 Swap Nodes in Pairs

| | |
|---|---|
| **Concept** | Linked List, Dummy node |
| **Problem** | Swap every two adjacent nodes (1->2->3->4 becomes 2->1->4->3). |

### Approach / Logic

1. Use dummy before head; `prev` points to the node before the current pair.
2. While we have at least two nodes: set `first = prev.next`, `second = prev.next.next`; rewire so second becomes first of the pair, first becomes second; move `prev` to `first`.
3. Set head from dummy.next.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```
def swap_pairs(self):
    dummy = Node(0)
    dummy.next = self.head
    prev = dummy
    while prev.next and prev.next.next:
        first = prev.next
        second = prev.next.next
        first.next = second.next
        second.next = first
        prev.next = second
```

```
            prev = first
        self.head = dummy.next
```

**Takeaways:** Dummy simplifies head update; draw the three pointers (prev, first, second) when wiring.

## 3.8 Binary Number in Linked List to Decimal

| | |
|---|---|
| **Concept** | Linked List, Binary representation |
| **Problem** | Head is MSB; convert the binary number represented by the list to decimal. |

### Approach / Logic

1. Traverse from head: for each bit, `decimal = decimal * 2 + bit`.
2. Return decimal.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(1) |

```
def binary_to_decimal(self):
    decimal = 0
    current = self.head
    while current:
        decimal = decimal * 2 + current.value
        current = current.next
    return decimal
```

### Example

- List: `1 -> 1 -> 0` → 6; `1 -> 0 -> 0 -> 0` → 8

**Takeaways:** Same as parsing a number in base 2 from left to right.

## 3.9 Doubly Linked List — Basic and Reverse

| | |
|---|---|
| **Concept** | Doubly Linked List, prev/next |

| | |
|---|---|
| **Problem** | Implement basic DLL operations and reverse the list in-place. |

**Reverse logic:** For each node, swap `prev` and `next`; then swap head and tail.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
        self.prev = None

class DoublyLinkedList:
    def __init__(self, value):
        new_node = Node(value)
        self.head = new_node
        self.tail = new_node
        self.length = 1

    def append(self, value):
        new_node = Node(value)
        if not self.head:
            self.head = self.tail = new_node
        else:
            self.tail.next = new_node
            new_node.prev = self.tail
            self.tail = new_node
        self.length += 1
        return True

    def reverse(self):
        current = self.head
        while current:
            current.prev, current.next = current.next, current.prev
            current = current.prev
        self.head, self.tail = self.tail, self.head
```

**Takeaways:** Reversing DLL is swapping prev/next at each node then swapping head/tail.

## 3.10 Palindrome (Doubly Linked List)

| | |
|---|---|
| **Concept** | Doubly Linked List, Two pointers from both ends |
| **Problem** | Check if a DLL reads the same forwards and backwards. |

**Approach:** Use `left = head` and `right = tail` ; compare values and move left forward, right backward until they meet or cross; if any pair differs, return False.

```
def is_palindrome(self):
    if self.length <= 1:
        return True
    left = self.head
    right = self.tail
    for _ in range(self.length // 2):
        if left.value != right.value:
            return False
        left = left.next
        right = right.prev
    return True
```

**Takeaways:** DLL allows O(1) access to both ends; palindrome check in O(n) time, O(1) space.

# 4. Stack

## 4.1 Stack Using List

| | |
|---|---|
| **Concept** | Stack (LIFO), List |
| **Problem** | Implement stack with push, pop, peek, is_empty, size. |

**Approach:** Use list; push = append, pop = pop from end, peek = last element.

| Time Complexity | Space Complexity |
|---|---|
| Push/Pop/Peek O(1) | O(n) |

```
class Stack:
    def __init__(self):
        self.stack_list = []

    def is_empty(self):
        return len(self.stack_list) == 0

    def peek(self):
        return self.stack_list[-1] if not self.is_empty() else None

    def size(self):
```

```
        return len(self.stack_list)

    def push(self, value):
        self.stack_list.append(value)

    def pop(self):
        return self.stack_list.pop() if not self.is_empty() else None
```

**Takeaways:** List end = top; all operations O(1).

## 4.2 Stack Using Linked List

| | |
|---|---|
| **Concept** | Stack, Linked list (head = top) |
| **Problem** | Implement stack with linked list; top of stack = head. |

**Approach:** Push: new node's next = head, head = new node. Pop: save head, head = head.next, return saved value.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class Stack:
    def __init__(self, value):
        new_node = Node(value)
        self.top = new_node
        self.height = 1

    def push(self, value):
        new_node = Node(value)
        new_node.next = self.top
        self.top = new_node
        self.height += 1
        return True

    def pop(self):
        if self.height == 0:
            return None
        temp = self.top
        self.top = self.top.next
        temp.next = None
```

```
        self.height -= 1
        return temp
```

**Takeaways:** No array resizing; each operation O(1).

## 4.3 Sort Stack (Auxiliary Stack)

| | |
|---|---|
| **Concept** | Stack, Sorting with temporary stack |
| **Problem** | Sort a stack so smallest is at top, using only one extra stack. |

**Approach / Logic**

1. Pop from original stack. While auxiliary stack is non-empty and its top is greater than the popped value, push auxiliary top back to original.
2. Push the popped value onto auxiliary.
3. After processing all, move all from auxiliary back to original (so smallest on top of original). Alternatively, keep sorted in auxiliary and copy back so smallest is at top.

*Clarification:* "Smallest at top" usually means we want the stack to have the smallest element on top. So we build the auxiliary stack with largest at bottom (when we push from original, we maintain auxiliary in sorted order with smallest at top). Then we move everything back to original so that when we pop we get smallest first — so original should have largest at top. If we want original to have smallest at top: after the while loop, auxiliary has smallest at bottom; push all from auxiliary to original so smallest ends at top.

Standard approach: while original not empty: temp = pop(original). While auxiliary not empty and peek(auxiliary) > temp: push(original, pop(auxiliary)). Push(auxiliary, temp). Then while auxiliary not empty: push(original, pop(auxiliary)). Now original has largest at top. To get smallest at top, we'd reverse. So "sort stack" with one auxiliary typically produces one order; for "smallest at top" we can either reverse at the end or use the opposite comparison. Here we'll state: sort so that when we pop repeatedly we get ascending order (smallest first). That means we want smallest at top. So after building auxiliary (which has smallest at bottom if we always push smaller elements and bounce larger ones back), we copy back to original. So: auxiliary will have smallest at bottom, largest at top. Then push all from auxiliary to original → original has largest at bottom, smallest at top. Good.

| Time Complexity | Space Complexity |
|---|---|
| $O(n^2)$ | $O(n)$ |

```python
def sort_stack(stack):
    additional = []
    while stack:
        temp = stack.pop()
        while additional and additional[-1] > temp:
            stack.append(additional.pop())
        additional.append(temp)
    while additional:
        stack.append(additional.pop())
```

**Takeaways:** Simulate "insertion sort" with two stacks.

## 4.4 Queue Using Two Stacks

| | |
|---|---|
| **Concept** | Queue (FIFO), Two stacks |
| **Problem** | Implement enqueue and dequeue using two stacks. |

**Approach (simple):** Enqueue: move all from stack1 to stack2, push new element to stack1, move all from stack2 back to stack1 (so front of queue is at top of stack1). Dequeue: pop from stack1.

| Time Complexity | Space Complexity |
|---|---|
| Enqueue O(n), Dequeue O(1) | O(n) |

```python
class MyQueue:
    def __init__(self):
        self.stack1 = []
        self.stack2 = []

    def enqueue(self, value):
        while self.stack1:
            self.stack2.append(self.stack1.pop())
        self.stack1.append(value)
        while self.stack2:
            self.stack1.append(self.stack2.pop())

    def dequeue(self):
        return self.stack1.pop() if self.stack1 else None

    def is_empty(self):
        return len(self.stack1) == 0
```

```
    def peek(self):
        return self.stack1[-1] if self.stack1 else None
```

**Takeaways:** Amortized O(1) enqueue is possible with lazy reversal (second stack used only for dequeue).

# 5. Queue

## 5.1 Queue Using Linked List

| | |
|---|---|
| **Concept** | Queue (FIFO), Singly linked list |
| **Problem** | Implement queue with enqueue (at rear) and dequeue (from front). |

**Approach:** Maintain `first` (front) and `last` (rear). Enqueue: new node at last, update last. Dequeue: remove first, update first.

| Time Complexity | Space Complexity |
|---|---|
| Enqueue O(1), Dequeue O(1) | O(n) |

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class Queue:
    def __init__(self, value):
        new_node = Node(value)
        self.first = self.last = new_node
        self.length = 1

    def enqueue(self, value):
        new_node = Node(value)
        if not self.first:
            self.first = self.last = new_node
        else:
            self.last.next = new_node
            self.last = new_node
        self.length += 1
        return True
```

```
    def dequeue(self):
        if self.length == 0:
            return None
        temp = self.first
        self.first = self.first.next
        temp.next = None
        self.length -= 1
        if self.length == 0:
            self.last = None
        return temp
```

**Takeaways:** First and last pointers give O(1) enqueue and dequeue.

# 6. Recursion

## 6.1 Factorial

| | |
|---|---|
| **Concept** | Recursion, Base case |
| **Problem** | Compute n! = n * (n-1)! with base case 1! = 1. |

**Approach:** Base case: n == 1 return 1. Else return n * factorial(n-1).

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(n) stack |

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

**Takeaways:** Always define base case and recurrence.

## 6.2 Call Stack (Concept)

Recursive calls push frames onto the call stack; return pops them. Example: `First()` calls `Two()` calls `Three()`; when `Three()` returns, `Two()` continues, then `One()`. Understanding the call stack helps with debugging and understanding recursion.

## 6.3 BST: Recursive Contains

| | |
|---|---|
| **Concept** | Binary Search Tree, Recursion |
| **Problem** | Check if value exists in BST using recursion. |

**Approach:** If node is None return False. If value equals node.value return True. If value < node.value search left, else search right.

```
def __r_contains(self, node, value):
    if node is None:
        return False
    if value == node.value:
        return True
    if value < node.value:
        return self.__r_contains(node.left, value)
    return self.__r_contains(node.right, value)

def r_contains(self, value):
    return self.__r_contains(self.root, value)
```

**Takeaways:** Recursive BST search mirrors the iterative version; one comparison per level.

## 6.4 BST: Minimum Value in Subtree

| | |
|---|---|
| **Concept** | BST, In-order successor (leftmost in right subtree) |
| **Problem** | Find minimum value in the tree rooted at given node (leftmost node). |

**Approach:** Go left until no left child; return that node's value.

```
def min_value(self, current_node):
    while current_node.left:
        current_node = current_node.left
    return current_node.value
```

**Takeaways:** In BST, minimum = leftmost; maximum = rightmost.

## 6.5 Sorted Array to Balanced BST

| | |
|---|---|
| **Concept** | BST, Recursion, Divide and conquer |
| **Problem** | Convert sorted array to a height-balanced BST (middle as root). |

**Approach:** Base: if left > right return None. Mid = (left+right)//2; create node(mid); node.left = build(left, mid-1); node.right = build(mid+1, right); return node.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(log n) recursion |

```
def sorted_list_to_bst(self, nums):
    self.root = self._build(nums, 0, len(nums) - 1)

def _build(self, nums, left, right):
    if left > right:
        return None
    mid = (left + right) // 2
    node = Node(nums[mid])
    node.left = self._build(nums, left, mid - 1)
    node.right = self._build(nums, mid + 1, right)
    return node
```

**Takeaways:** Middle element as root keeps the tree balanced.

## 6.6 Invert Binary Tree

| | |
|---|---|
| **Concept** | Binary Tree, Recursion |
| **Problem** | Mirror the tree (swap left and right for every node). |

**Approach:** Base: if node is None return None. Swap node.left and node.right; recursively invert left and right; return node.

```
def invert(self):
    self.root = self._invert(self.root)

def _invert(self, node):
    if node is None:
```

```
            return None
        node.left, node.right = node.right, node.left
        self._invert(node.left)
        self._invert(node.right)
        return node
```

**Takeaways:** Post-order or pre-order both work; swap then recurse.

## 6.7 BST: Delete Node

| | |
|---|---|
| **Concept** | BST, Recursion, Three cases |
| **Problem** | Delete a node with given value from BST. |

**Approach:** If value < node.value recurse left; if value > node.value recurse right. If found: (1) no children → return None; (2) one child → return that child; (3) two children → replace node.value with min of right subtree, then delete that min from right subtree.

```
def __delete_node(self, node, value):
    if node is None:
        return None
    if value < node.value:
        node.left = self.__delete_node(node.left, value)
    elif value > node.value:
        node.right = self.__delete_node(node.right, value)
    else:
        if not node.left and not node.right:
            return None
        if not node.left:
            return node.right
        if not node.right:
            return node.left
        min_val = self.min_value(node.right)
        node.value = min_val
        node.right = self.__delete_node(node.right, min_val)
    return node

def delete_node(self, value):
    self.root = self.__delete_node(self.root, value)
```

**Takeaways:** Two-children case: replace with inorder successor (min of right subtree) or predecessor (max of left subtree).

# 7. Sorting Algorithms

## 7.1 Bubble Sort

| | |
|---|---|
| **Concept** | Array, Comparison sort |
| **Problem** | Sort by repeatedly swapping adjacent elements if they are in wrong order; largest "bubbles" to end each pass. |

**Approach:** Outer loop from end to 1; inner loop 0 to i-1; if arr[j] > arr[j+1] swap. After each pass, largest is at position i.

| Time Complexity | Space Complexity |
|---|---|
| $O(n^2)$ | O(1) |

```
def bubble_sort(arr):
    for i in range(len(arr) - 1, 0, -1):
        for j in range(i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

**Takeaways:** Stable, in-place; rarely used for large data.

## 7.2 Selection Sort

| | |
|---|---|
| **Concept** | Array, Comparison sort |
| **Problem** | Repeatedly select the minimum from unsorted part and put at the front. |

**Approach:** For i from 0 to n-2: find min index in range [i, n); swap arr[i] with arr[min_index].

| Time Complexity | Space Complexity |
|---|---|
| $O(n^2)$ | O(1) |

```
def selection_sort(arr):
    for i in range(len(arr) - 1):
        min_idx = i
```

```
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

**Takeaways:** Unstable (e.g. 4a 4b 2 → 2 4b 4a); minimal swaps.

## 7.3 Insertion Sort

| | |
|---|---|
| **Concept** | Array, Comparison sort |
| **Problem** | Build sorted portion from left; for each new element, insert it into the correct position in the sorted part. |

**Approach:** For i from 1 to n-1: save arr[i], shift elements to the right until we find position for temp, then write temp.

| Time Complexity | Space Complexity |
|---|---|
| $O(n^2)$ worst, O(n) best | O(1) |

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        temp = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > temp:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = temp
    return arr
```

**Takeaways:** Good for small or nearly sorted data; stable.

## 7.4 Merge Sort

| | |
|---|---|
| **Concept** | Divide and conquer, Recursion |
| **Problem** | Split array in half, sort halves, merge two sorted halves. |

**Approach:** Base: length 1 return. Mid = n//2; left = merge_sort(left half); right = merge_sort(right half); return merge(left, right). Merge: two pointers, take smaller, then append remainder.

| Time Complexity | Space Complexity |
|---|---|
| O(n log n) | O(n) |

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
```

**Takeaways:** Stable, predictable O(n log n); extra space for merge.

## 7.5 Quick Sort

| | |
|---|---|
| **Concept** | Divide and conquer, Pivot, Partition |
| **Problem** | Choose pivot (e.g. first), partition so smaller elements are left, larger right; recursively sort left and right. |

**Approach:** Partition: pivot = arr[start]; swap_index = start; for i in [start+1, end): if arr[i] < pivot: swap_index++; swap arr[i] and arr[swap_index]; finally swap arr[start] and arr[swap_index]; return swap_index. Then quicksort(left, pivot_idx-1) and quicksort(pivot_idx+1, right).

| Time Complexity | Space Complexity |
|---|---|
| O(n log n) avg, O($n^2$) worst | O(log n) recursion |

```
def pivot(arr, start, end):
    pivot_idx = start
    swap_idx = start
    for i in range(start + 1, end):
        if arr[i] < arr[pivot_idx]:
            swap_idx += 1
            arr[swap_idx], arr[i] = arr[i], arr[swap_idx]
    arr[pivot_idx], arr[swap_idx] = arr[swap_idx], arr[pivot_idx]
    return swap_idx

def quick_sort_helper(arr, left, right):
    if left < right:
        p = pivot(arr, left, right)
        quick_sort_helper(arr, left, p)
        quick_sort_helper(arr, p + 1, right)
    return arr

def quick_sort(arr):
    return quick_sort_helper(arr, 0, len(arr))
```

**Takeaways:** In-place; pivot choice affects performance; random pivot avoids worst case on sorted input.

# 8. Searching Algorithms

## 8.1 Binary Search (Sorted Array)

| | |
|---|---|
| **Concept** | Sorted array, Divide and conquer |
| **Problem** | Find index of target in sorted array; return -1 if not present. |

**Approach:** Maintain left and right; while left <= right: mid = (left+right)//2; if arr[mid] == target return mid; if target < arr[mid] right = mid-1; else left = mid+1. Return -1.

| Time Complexity | Space Complexity |
|---|---|
| O(log n) | O(1) |

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        if target < arr[mid]:
            right = mid - 1
        else:
            left = mid + 1
    return -1
```

**Example**

- Input: `[1, 3, 5, 7, 9]`, target `5` → 2

**Takeaways:** Only works on sorted array; off-by-one with left/right and mid±1.

# 9. Trees

## 9.1 Binary Search Tree — Insert and Contains

| | |
|---|---|
| **Concept** | Binary Search Tree |
| **Problem** | Insert value in BST; check if value exists (contains). |

**Approach:** Insert: if root None, new root. Else traverse: if value < node go left, if value > node go right; when child is None, attach new node. Contains: traverse similarly; return True if found, False if reach None.

| Time Complexity | Space Complexity |
|---|---|
| O(h) average O(log n), worst O(n) | O(1) iterative |

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None
```

```python
    def insert(self, value):
        new_node = Node(value)
        if not self.root:
            self.root = new_node
            return True
        temp = self.root
        while True:
            if value == temp.value:
                return False
            if value < temp.value:
                if not temp.left:
                    temp.left = new_node
                    return True
                temp = temp.left
            else:
                if not temp.right:
                    temp.right = new_node
                    return True
                temp = temp.right

    def contains(self, value):
        temp = self.root
        while temp:
            if value == temp.value:
                return True
            if value < temp.value:
                temp = temp.left
            else:
                temp = temp.right
        return False
```

**Takeaways:** BST property: left < root < right; iterative insert/contains avoid recursion stack.

## 9.2 Tree Traversal — BFS and DFS (Pre, In, Post)

| | |
|---|---|
| **Concept** | BFS (level-order), DFS (pre/in/post-order) |
| **Problem** | Traverse all nodes: level-by-level; pre (root, left, right); in (left, root, right); post (left, right, root). |

**BFS:** Queue; enqueue root; while queue not empty: dequeue, process, enqueue left and right.
**DFS Pre:** Process node, recurse left, recurse right.

**DFS In:** Recurse left, process node, recurse right (BST gives sorted order).
**DFS Post:** Recurse left, recurse right, process node.

```
def BFS(self):
    if not self.root:
        return []
    result = []
    queue = [self.root]
    while queue:
        node = queue.pop(0)
        result.append(node.value)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
    return result

def dfs_in_order(self):
    result = []
    def traverse(node):
        if not node:
            return
        traverse(node.left)
        result.append(node.value)
        traverse(node.right)
    traverse(self.root)
    return result
```

**Takeaways:** In-order on BST = sorted; BFS uses queue; DFS uses stack (recursion or explicit).

## 9.3 Kth Smallest in BST

| | |
|---|---|
| **Concept** | BST, In-order traversal |
| **Problem** | Find the k-th smallest element (1-indexed) in BST. |

**Approach:** In-order traversal (left, node, right) visits in ascending order. Iterative: use stack, go left until None, pop and decrement k; when k==0 return value; else go right.

| Time Complexity | Space Complexity |
|---|---|
| O(h + k) | O(h) |

```python
def kth_smallest(self, k):
    stack = []
    current = self.root
    while stack or current:
        while current:
            stack.append(current)
            current = current.left
        current = stack.pop()
        k -= 1
        if k == 0:
            return current.value
        current = current.right
    return None
```

**Takeaways:** In-order gives sorted order; iterative avoids full traversal when k is small.

### 9.4 Validate BST

| | |
|---|---|
| **Concept** | BST, In-order |
| **Problem** | Check if the tree is a valid BST (in-order is strictly increasing). |

**Approach:** Do in-order traversal; if any value is ≤ previous value, return False. Else True.

```python
def is_valid_bst(self):
    values = self.dfs_in_order()  # or inline in-order
    for i in range(1, len(values)):
        if values[i] <= values[i - 1]:
            return False
    return True
```

**Takeaways:** Valid BST iff in-order sequence is strictly increasing. Alternative: pass (min, max) per node.

## 10. Graphs

### 10.1 Graph — Adjacency List

| | |
|---|---|
| **Concept** | Graph, Adjacency list |
| **Problem** | Implement undirected graph with add_vertex, add_edge, remove_edge, remove_vertex. |

**Approach:** Store as dict: vertex -> list of neighbors. add_edge: append each to the other's list. remove_vertex: remove vertex from all neighbors' lists, then delete vertex.

| Time Complexity | Space Complexity |
|---|---|
| add_edge O(1), remove_vertex O(degree) | O(V + E) |

```
class Graph:
    def __init__(self):
        self.adj_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []
            return True
        return False

    def add_edge(self, v1, v2):
        if v1 in self.adj_list and v2 in self.adj_list:
            self.adj_list[v1].append(v2)
            self.adj_list[v2].append(v1)
            return True
        return False

    def remove_edge(self, v1, v2):
        if v1 in self.adj_list and v2 in self.adj_list:
            try:
                self.adj_list[v1].remove(v2)
                self.adj_list[v2].remove(v1)
            except ValueError:
                pass
            return True
        return False

    def remove_vertex(self, vertex):
        if vertex not in self.adj_list:
            return False
        for neighbor in self.adj_list[vertex]:
            self.adj_list[neighbor].remove(vertex)
```

```
        del self.adj_list[vertex]
        return True
```

**Takeaways:** Adjacency list is flexible and space-efficient for sparse graphs.

# 11. Dynamic Programming

## 11.1 Fibonacci — Recursive, Memoization, Tabulation, Optimized

| | |
|---|---|
| **Concept** | DP, Memoization (top-down), Tabulation (bottom-up) |
| **Problem** | Compute F(n) where F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2). |

### Approaches

1. **Naive recursion:** Direct recurrence; O(2^n) time, many repeated subcalls.
2. **Memoization:** Cache results in dict; before recurring check cache; after computing store in cache. O(n) time, O(n) space.
3. **Tabulation:** Fill table from 0 to n; table[i] = table[i-1] + table[i-2]. O(n) time, O(n) space.
4. **Space-optimized:** Keep only two variables (prev, curr); update in loop. O(n) time, O(1) space.

```
# Memoization (top-down)
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

# Tabulation (bottom-up)
def fib_tab(n):
    if n <= 1:
        return n
    dp = [0, 1]
    for i in range(2, n + 1):
        dp.append(dp[i - 1] + dp[i - 2])
    return dp[n]

# Space-optimized
```

```python
def fib_optimized(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b
```

**Takeaways:** Overlapping subproblems → cache; memoization = recursion + cache; tabulation = iterative fill; often possible to reduce to O(1) extra space.

# 12. Miscellaneous Important Problems

## 12.1 Two Sum

| | |
|---|---|
| **Concept** | Array, Hash map |
| **Problem** | Find two indices such that nums[i] + nums[j] = target (one solution). |

**Approach:** One pass: for each num, check if (target - num) is in a map of value -> index; if yes return [map[target-num], i]; else add num -> i to map.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(n) |

```python
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

**Takeaways:** Hash map stores “what we need” (complement) or “what we have” (value -> index).

## 12.2 Group Anagrams

| | |
|---|---|
| **Concept** | String, Hash map (signature as key) |
| **Problem** | Group strings that are anagrams of each other. |

**Approach:** Use a canonical signature for each string (e.g. sorted string or tuple of character counts). Map signature -> list of strings; return all values.

| Time Complexity | Space Complexity |
|---|---|
| O(n * k log k) or O(n * k) with count | O(n * k) |

*k = max string length; count signature avoids sort*

```
def group_anagrams(strs):
    groups = {}
    for s in strs:
        key = tuple(sorted(s))  # or use count tuple for O(n*k)
        if key not in groups:
            groups[key] = []
        groups[key].append(s)
    return list(groups.values())
```

**Takeaways:** Choosing a canonical key (sorted or count) groups anagrams.

## 12.3 Longest Consecutive Sequence

| | |
|---|---|
| **Concept** | Array, Hash set |
| **Problem** | Find length of longest consecutive integer sequence (each element in array). |

**Approach:** Put all numbers in a set. For each num, if num-1 is not in set (so num is start of a streak), count forward (num, num+1, ...) and update longest.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(n) |

```
def longest_consecutive(nums):
    num_set = set(nums)
    longest = 0
    for num in num_set:
```

```
        if num - 1 not in num_set:
            current = num
            length = 0
            while current in num_set:
                length += 1
                current += 1
            longest = max(longest, length)
    return longest
```

### Example

- Input: `[100, 4, 200, 1, 3, 2]` → 4 (sequence 1,2,3,4)

**Takeaways:** Only start counting from the smallest element of each streak (num-1 not in set).

## 12.4 Subarray Sum Equals Target (Prefix Sum + Hash)

| | |
|---|---|
| **Concept** | Array, Prefix sum, Hash map |
| **Problem** | Find contiguous subarray that sums to target; return start and end indices. |

**Approach:** prefix_sum[i] = sum of nums[0..i]. We need prefix_sum[j] - prefix_sum[i-1] = target, i.e. prefix_sum[i-1] = prefix_sum[j] - target. One pass: maintain current sum and a map (sum -> latest index). For each j, if (current_sum - target) in map, return [map[current_sum - target] + 1, j]. Else set map[current_sum] = j. Initialize map with 0 -> -1 for subarrays starting at 0.

| Time Complexity | Space Complexity |
|---|---|
| O(n) | O(n) |

```
def subarray_sum(nums, target):
    prefix = {0: -1}
    current_sum = 0
    for i, num in enumerate(nums):
        current_sum += num
        need = current_sum - target
        if need in prefix:
            return [prefix[need] + 1, i]
        prefix[current_sum] = i
    return []
```

**Takeaways:** prefix_sum[j] - prefix_sum[i-1] = sum(i..j); map stores prefix sums seen so far.

## 12.5 Min-Heap and Max-Heap (Array-Based)

| | |
|---|---|
| **Concept** | Heap, Complete binary tree in array |
| **Problem** | Implement min-heap (min at root) and max-heap with insert and remove root. |

**Approach:** Store in array; for index i, left = 2*i+1, right* = *2*i+2, parent = (i-1)//2. Insert: append at end, then bubble up (swap with parent while smaller (min) or larger (max)). Remove: replace root with last element, then sink down (swap with smaller child (min) or larger (max) until heap property holds).

| Time Complexity | Space Complexity |
|---|---|
| Insert O(log n), Remove O(log n) | O(n) |

```
class MinHeap:
    def __init__(self):
        self.heap = []

    def _parent(self, i):
        return (i - 1) // 2

    def _left(self, i):
        return 2 * i + 1

    def _right(self, i):
        return 2 * i + 2

    def _swap(self, i, j):
        self.heap[i], self.heap[j] = self.heap[j], self.heap[i]

    def insert(self, value):
        self.heap.append(value)
        i = len(self.heap) - 1
        while i > 0 and self.heap[i] < self.heap[self._parent(i)]:
            self._swap(i, self._parent(i))
            i = self._parent(i)

    def _sink_down(self, i):
        n = len(self.heap)
        while True:
            left = self._left(i)
            right = self._right(i)
            smallest = i
```

```
            if left < n and self.heap[left] < self.heap[smallest]:
                smallest = left
            if right < n and self.heap[right] < self.heap[smallest]:
                smallest = right
            if smallest == i:
                break
            self._swap(i, smallest)
            i = smallest

    def remove(self):
        if not self.heap:
            return None
        if len(self.heap) == 1:
            return self.heap.pop()
        min_val = self.heap[0]
        self.heap[0] = self.heap.pop()
        self._sink_down(0)
        return min_val
```

**Max-Heap:** Same with comparisons reversed (larger wins for bubble-up and sink-down).

**Takeaways:** Array representation avoids pointers; used for priority queue, heapsort, and k-th smallest/largest.

*End of handbook. Use this document for revision and export to PDF as needed.*